

Computational Physics

A Comprehensive Guide to Numerical Methods

From Integration to Quantum Monte Carlo

Contents

1	Introduction to Computational Physics	5
1.1	Why Computational Methods?	5
1.2	Sources of Error	5
1.3	Order of Convergence	6
1.4	Python as a Computational Tool	6
2	Numerical Integration	7
2.1	The Trapezoidal Rule	7
2.2	Simpson's Rule	8
2.3	Error Convergence	8
2.4	Richardson Extrapolation	9
2.5	Gaussian Quadrature	9
2.6	Special Functions via Integration	11
2.6.1	Bessel Functions	11
2.6.2	Fresnel Integrals	12
3	Linear Algebra and Systems of Equations	13
3.1	Gaussian Elimination	13
3.2	Partial Pivoting	14
3.3	LU Decomposition	14
3.4	Conditioning and Stability	14
4	Fourier Analysis and Spectral Methods	17
4.1	The Discrete Fourier Transform	17
4.2	Frequency Resolution and Aliasing	18
4.3	Power Spectrum	18
4.4	Fourier Series and Gibbs Phenomenon	19
5	Ordinary Differential Equations	21
5.1	Euler's Method	21
5.2	The Runge-Kutta Family	21
5.3	Systems of ODEs	22
5.4	The Damped Driven Oscillator	23

6	Orbital Mechanics	25
6.1	Newtonian Gravity	25
6.2	General Relativistic Correction	26
6.3	Symplectic Integrators	27
7	Adaptive Methods and Advanced ODE Solvers	29
7.1	Step Doubling	29
7.2	The Bulirsch-Stoer Method	31
8	Elliptic Partial Differential Equations	33
8.1	Laplace's Equation	33
8.2	Finite Difference Discretization	33
8.3	Iterative Solvers	33
8.3.1	Jacobi Method	33
8.3.2	Gauss-Seidel Method	34
8.3.3	Successive Over-Relaxation (SOR)	34
9	Hyperbolic Partial Differential Equations	37
9.1	The Advection Equation	37
9.2	Finite Difference Schemes	37
9.2.1	FTCS Scheme (Unstable!)	37
9.2.2	Lax-Friedrichs Scheme	37
9.2.3	Leapfrog Scheme	38
9.2.4	Lax-Wendroff Scheme	38
9.3	The CFL Condition	38
9.4	Burgers' Equation	38
10	Monte Carlo Methods	41
10.1	Monte Carlo Integration	41
10.2	Importance Sampling	42
10.3	Random Walks and Transport	42
11	Markov Chain Monte Carlo and Statistical Mechanics	45
11.1	The Boltzmann Distribution	45
11.2	The Metropolis Algorithm	45
11.3	Quantum Systems at Finite Temperature	47
11.4	Practical Considerations	47
A	Mathematical Reference	49
A.1	Taylor Series	49
A.2	Linear Algebra Essentials	49
A.3	Probability and Statistics	49
A.4	Useful Integrals	50
A.5	Physical Constants	50

Chapter 1

Introduction to Computational Physics

1.1 Why Computational Methods?

Many problems in physics have no closed-form analytical solution. From the three-body problem in celestial mechanics to turbulent fluid flows, the equations governing physical systems are often nonlinear partial differential equations that resist exact solution. Computational physics bridges this gap by developing numerical algorithms to approximate solutions with controlled accuracy.

The core philosophy is straightforward: replace continuous operations (derivatives, integrals) with discrete approximations, then use a computer to carry out the resulting arithmetic. The art lies in choosing approximations that are accurate, stable, and efficient.

1.2 Sources of Error

Every numerical computation carries error from multiple sources.

Definition 1.1 (Roundoff Error). Roundoff error arises because computers represent real numbers with finite precision. In IEEE 754 double-precision arithmetic, approximately 16 significant decimal digits are stored, giving machine epsilon $\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$.

Definition 1.2 (Truncation Error). Truncation error arises from approximating a mathematical operation. For instance, approximating a derivative by a finite difference:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \tag{1.1}$$

incurs an error proportional to h that vanishes only in the limit $h \rightarrow 0$.

There is a fundamental tension between these two: decreasing h reduces truncation error but eventually amplifies roundoff error. Finding the optimal balance is a recurring theme.

1.3 Order of Convergence

An algorithm has **order of convergence** p if its error satisfies

$$|E(h)| \leq C h^p \tag{1.2}$$

for some constant C independent of h . On a log-log plot of error versus step size, this appears as a line with slope p . Higher-order methods achieve a given accuracy with fewer function evaluations, but they may involve more complex formulas and can be less robust.

1.4 Python as a Computational Tool

Throughout this text we use Python with NumPy for numerical work and Matplotlib for visualization. A typical setup begins with:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

NumPy provides efficient array operations and linear algebra routines, while Matplotlib offers publication-quality plotting. Together they form the standard toolkit for computational physics in Python.

Chapter 2

Numerical Integration

Numerical integration (quadrature) is one of the most fundamental tasks in computational physics. We seek to evaluate

$$I = \int_a^b f(x) dx \quad (2.1)$$

when f has no convenient antiderivative, or when f is known only at discrete points.

2.1 The Trapezoidal Rule

The simplest approach is to approximate $f(x)$ as piecewise linear. Dividing $[a, b]$ into N panels of width $h = (b - a)/N$ with nodes $x_k = a + kh$, the trapezoidal rule gives:

$$I_T = h \left[\frac{f(a)}{2} + \sum_{k=1}^{N-1} f(x_k) + \frac{f(b)}{2} \right] \quad (2.2)$$

The error of the trapezoidal rule is:

$$E_T = -\frac{(b-a)}{12} h^2 f''(\xi), \quad \xi \in [a, b] \quad (2.3)$$

so the method is **second-order**: $|E_T| = O(h^2)$.

```
1 def trapezoidal(f, a, b, N):  
2     """Composite trapezoidal rule with N panels."""  
3     h = (b - a) / N  
4     x = np.linspace(a, b, N + 1)  
5     y = f(x)  
6     return h * (y[0]/2 + np.sum(y[1:-1]) + y[-1]/2)
```

2.2 Simpson's Rule

By fitting parabolas through successive triplets of points, we obtain Simpson's rule. For N panels (with N even):

$$I_S = \frac{h}{3} \left[f(a) + 4 \sum_{k=1,3,5,\dots}^{N-1} f(x_k) + 2 \sum_{k=2,4,6,\dots}^{N-2} f(x_k) + f(b) \right] \quad (2.4)$$

The error is:

$$E_S = -\frac{(b-a)}{180} h^4 f^{(4)}(\xi) \quad (2.5)$$

making Simpson's rule **fourth-order**: $|E_S| = O(h^4)$.

```

1 def simpsons(f, a, b, N):
2     """Composite Simpson's rule. N must be even."""
3     h = (b - a) / N
4     x = np.linspace(a, b, N + 1)
5     y = f(x)
6     return h/3 * (y[0] + 4*np.sum(y[1:-1:2])
7                   + 2*np.sum(y[2:-2:2]) + y[-1])

```

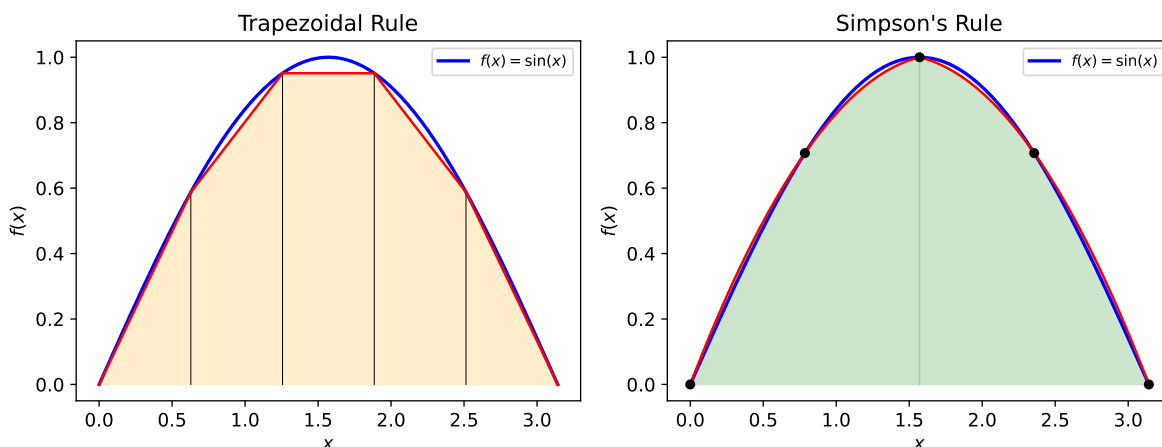


Figure 2.1: Comparison of the trapezoidal rule (left) and Simpson's rule (right) for $f(x) = \sin(x)$. The trapezoidal rule uses linear interpolation between nodes, while Simpson's rule fits parabolic arcs through triplets of points, capturing curvature more accurately.

2.3 Error Convergence

The theoretical convergence rates can be verified numerically. Computing $\int_0^\pi \sin(x) dx = 2$ with increasing N and plotting the error on a log-log scale reveals the expected slopes of -2 (trapezoidal) and -4 (Simpson).

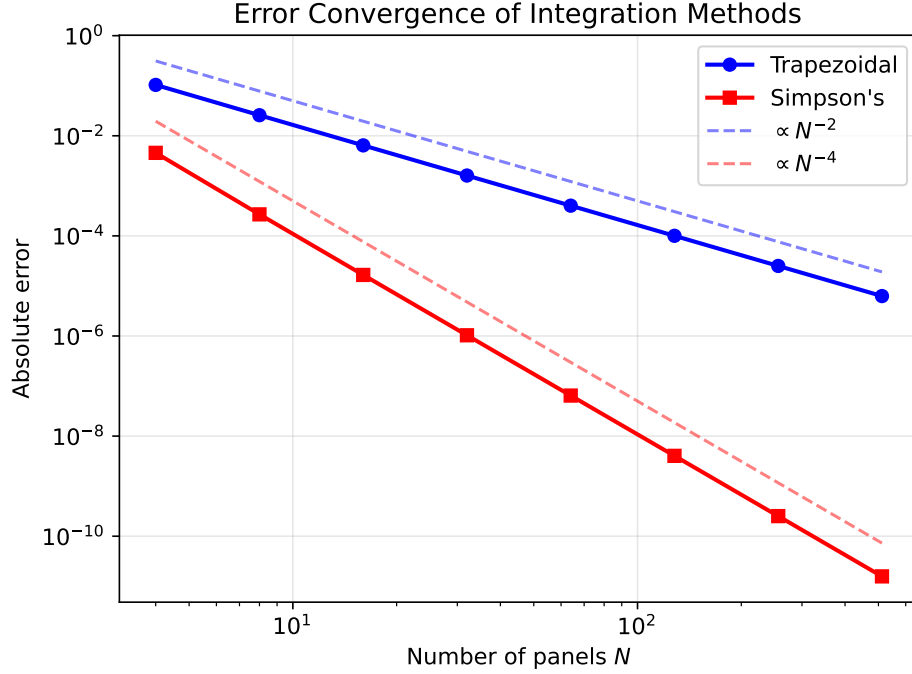


Figure 2.2: Error convergence for the trapezoidal and Simpson's rules applied to $\int_0^\pi \sin(x) dx$. The slopes confirm $O(h^2)$ and $O(h^4)$ convergence respectively.

2.4 Richardson Extrapolation

Given two estimates $I(h)$ and $I(h/2)$ from a method of order p , we can eliminate the leading error term:

$$I_{\text{improved}} = \frac{2^p I(h/2) - I(h)}{2^p - 1} \quad (2.6)$$

This yields an estimate of order $p + 1$ (or higher) at no additional function evaluations. For the trapezoidal rule ($p = 2$), this recovers Simpson's rule.

A practical use of Richardson extrapolation is **error estimation**: the difference $|I(h) - I(h/2)|$ provides an estimate of the error in $I(h/2)$, useful for adaptive algorithms.

2.5 Gaussian Quadrature

The methods above use equally-spaced nodes. Gaussian quadrature achieves higher accuracy by choosing both nodes and weights optimally:

$$\int_{-1}^1 f(x) dx \approx \sum_{k=1}^n w_k f(x_k) \quad (2.7)$$

where the n nodes x_k are roots of the Legendre polynomial $P_n(x)$, and the weights are:

$$w_k = \frac{2}{(1 - x_k^2)[P'_n(x_k)]^2} \quad (2.8)$$

Gauss-Legendre quadrature with n points integrates polynomials of degree up to $2n - 1$ exactly. For smooth functions, convergence is typically exponential rather than polynomial.

To integrate over a general interval $[a, b]$, apply the change of variables $x = \frac{b-a}{2}t + \frac{a+b}{2}$:

$$\int_a^b f(x) dx = \frac{b-a}{2} \sum_{k=1}^n w_k f\left(\frac{b-a}{2}x_k + \frac{a+b}{2}\right) \quad (2.9)$$

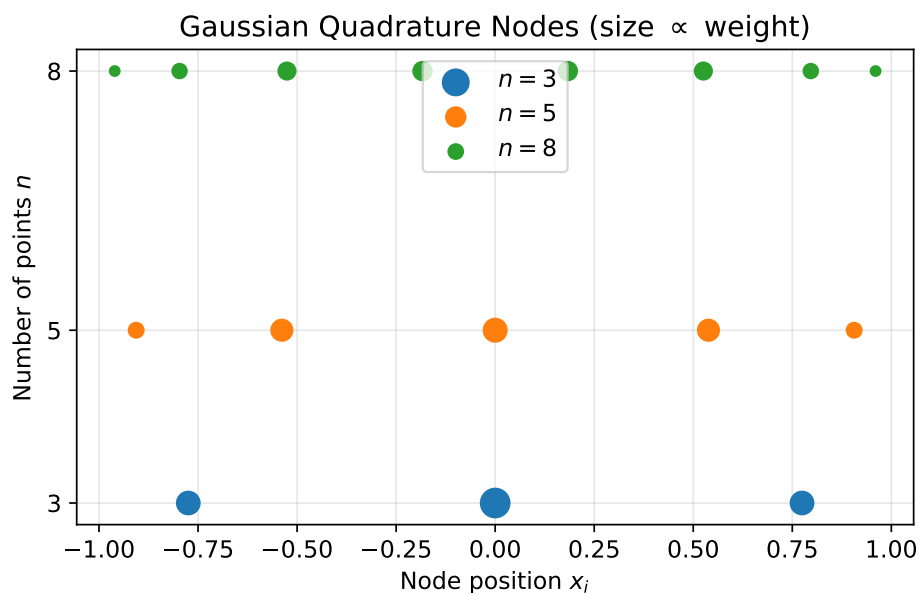


Figure 2.3: Gaussian quadrature nodes on $[-1, 1]$ for $n = 3, 5, 8$ points. Marker size is proportional to the weight w_k . Nodes cluster near the endpoints, where polynomial interpolation is most sensitive.

```

1 def gaussxw(N):
2     """Compute Gauss-Legendre nodes and weights."""
3     # Initial guesses for roots
4     a = np.linspace(3, 4*N - 1, N) / (4*N + 2)
5     x = np.cos(np.pi * a + 1/(8*N*N*np.tan(a)))
6
7     # Newton's method refinement
8     epsilon = 1e-15
9     delta = 1.0
10    while delta > epsilon:
11        p0 = np.ones(N)
12        p1 = np.copy(x)
13        for k in range(1, N):
14            p0, p1 = p1, ((2*k+1)*x*p1 - k*p0) / (k+1)
15        dp = (N+1) * (p0 - x*p1) / (1 - x*x)
16        dx = p1 / dp
17        x -= dx

```

```

18     delta = max(abs(dx))
19
20     w = 2 * (N+1)**2 / (N*N * (1-x*x) * dp*dp)
21     return x, w

```

2.6 Special Functions via Integration

Many special functions in physics are defined by integrals. Computing them numerically connects quadrature directly to mathematical physics.

2.6.1 Bessel Functions

The Bessel function of the first kind $J_n(x)$ arises in problems with cylindrical symmetry (vibrating drums, electromagnetic waveguides, diffraction). Its integral representation is:

$$J_n(x) = \frac{1}{\pi} \int_0^\pi \cos(n\theta - x \sin \theta) d\theta \quad (2.10)$$

This integral is well-suited to Gaussian quadrature since the integrand is smooth and periodic.

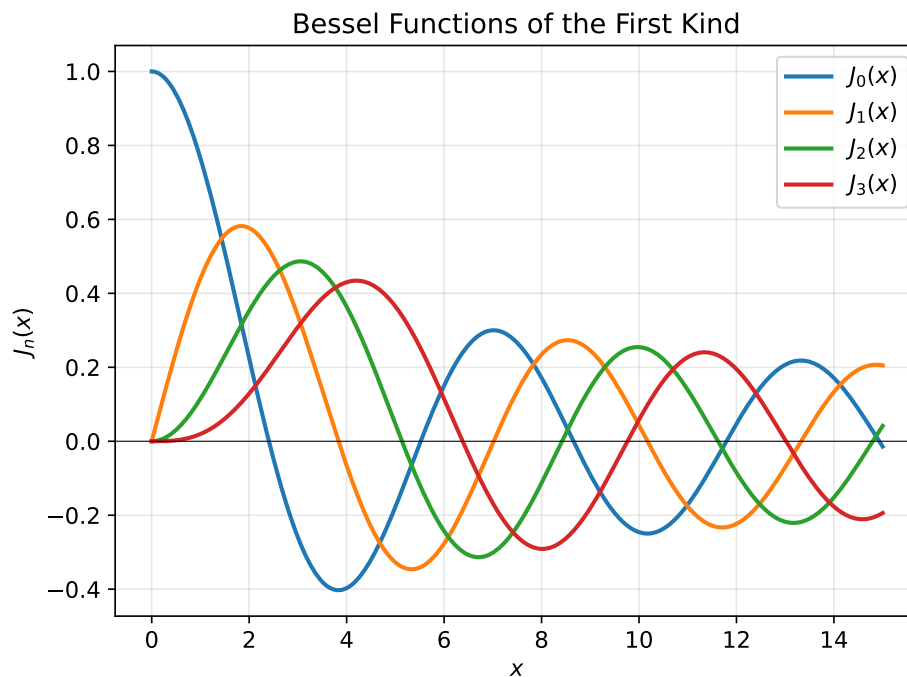


Figure 2.4: Bessel functions $J_0(x)$ through $J_3(x)$ computed via numerical integration of the integral representation. These oscillatory functions appear throughout wave physics.

2.6.2 Fresnel Integrals

The Fresnel integrals arise in diffraction theory:

$$C(t) = \int_0^t \cos\left(\frac{\pi u^2}{2}\right) du, \quad S(t) = \int_0^t \sin\left(\frac{\pi u^2}{2}\right) du \quad (2.11)$$

As $t \rightarrow \infty$, both integrals converge to $\frac{1}{2}$. Plotting $S(t)$ versus $C(t)$ yields the **Cornu spiral**, a beautiful curve used in optical design.

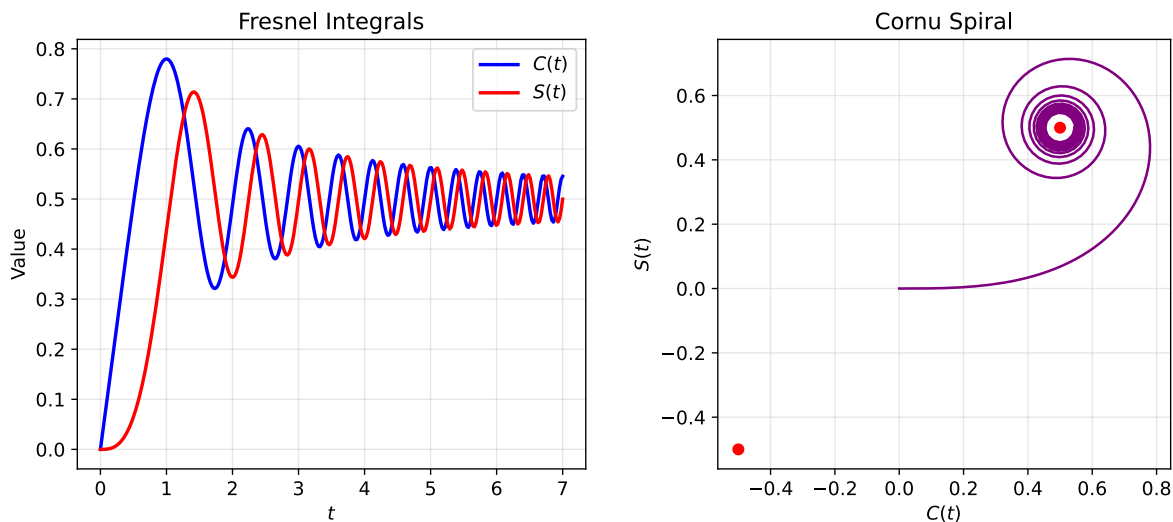


Figure 2.5: Left: Fresnel integrals $C(t)$ and $S(t)$ as functions of t . Both oscillate with decreasing amplitude, converging to $\frac{1}{2}$. Right: The Cornu spiral formed by plotting S versus C , spiraling inward toward the points $(\pm\frac{1}{2}, \pm\frac{1}{2})$.

Chapter 3

Linear Algebra and Systems of Equations

Systems of linear equations $A\mathbf{x} = \mathbf{b}$ appear everywhere in computational physics: from discretized differential equations to least-squares fitting. Efficient, stable algorithms for solving them are essential.

3.1 Gaussian Elimination

The classical algorithm reduces A to upper-triangular form through elementary row operations, then solves by back-substitution.

Given an $n \times n$ system, forward elimination proceeds column by column. For column j , we eliminate all entries below the diagonal by subtracting appropriate multiples of row j from subsequent rows:

$$a_{ij}^{(j)} \leftarrow a_{ij}^{(j-1)} - \frac{a_{ij}^{(j-1)}}{a_{jj}^{(j-1)}} \cdot a_{jk}^{(j-1)}, \quad i > j \quad (3.1)$$

Back-substitution then yields:

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=i+1}^n a_{ij} x_j \right), \quad i = n, n-1, \dots, 1 \quad (3.2)$$

The operation count is $O(n^3)$ for elimination and $O(n^2)$ for back-substitution.

```
1 def gauss_eliminate(A, b):
2     """Solve Ax = b by Gaussian elimination with partial pivoting.
3     """
4     n = len(b)
5     Ab = np.column_stack([A.astype(float), b.astype(float)])
6
7     for j in range(n):
8         # Partial pivoting
9         max_row = j + np.argmax(np.abs(Ab[j:, j]))
10        Ab[[j, max_row]] = Ab[[max_row, j]]
```

```

10
11     # Elimination
12     for i in range(j+1, n):
13         factor = Ab[i, j] / Ab[j, j]
14         Ab[i, j:] -= factor * Ab[j, j:]
15
16     # Back-substitution
17     x = np.zeros(n)
18     for i in range(n-1, -1, -1):
19         x[i] = (Ab[i, -1] - np.dot(Ab[i, i+1:n], x[i+1:])) / Ab[i, i]
20     return x

```

3.2 Partial Pivoting

Without pivoting, Gaussian elimination fails when a diagonal element is zero and becomes numerically unstable when it is small. **Partial pivoting** swaps the current row with the row below it that has the largest absolute value in the current column. This ensures the multipliers $|m_{ij}| \leq 1$, dramatically improving stability.

3.3 LU Decomposition

Rather than solving $A\mathbf{x} = \mathbf{b}$ directly, we can factor $A = LU$ where L is lower-triangular (with ones on the diagonal) and U is upper-triangular. Then solving becomes two triangular solves:

$$L\mathbf{y} = \mathbf{b} \quad (\text{forward substitution}), \quad U\mathbf{x} = \mathbf{y} \quad (\text{back-substitution}) \quad (3.3)$$

The advantage is that the factorization ($O(n^3)$) is done once, and each subsequent right-hand side requires only $O(n^2)$ work. This is valuable when solving many systems with the same matrix A .

3.4 Conditioning and Stability

The **condition number** $\kappa(A) = \|A\| \cdot \|A^{-1}\|$ measures how sensitive the solution is to perturbations in the data. A matrix with $\kappa(A) \approx 10^k$ loses roughly k digits of accuracy in the solution.

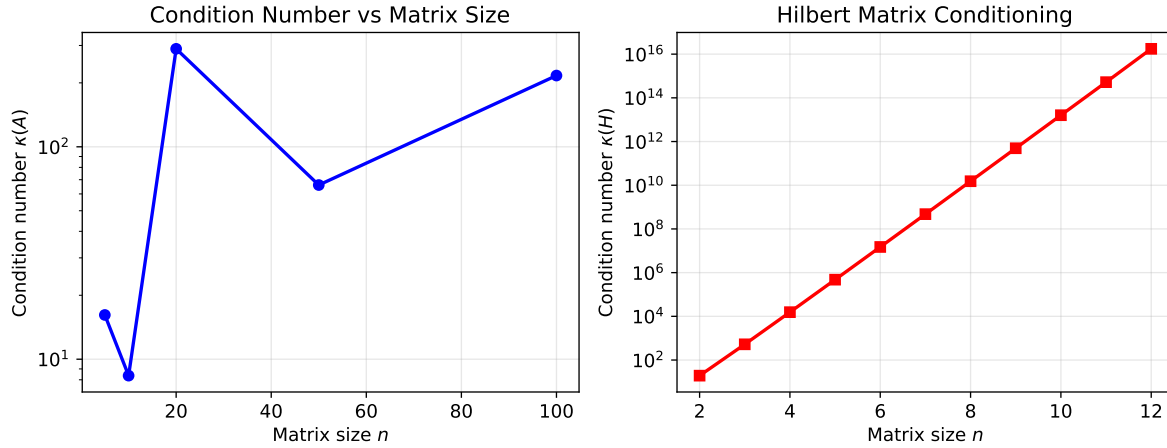


Figure 3.1: Left: condition numbers of random matrices grow modestly with size. Right: the Hilbert matrix $H_{ij} = 1/(i + j - 1)$ is notoriously ill-conditioned, with κ growing super-exponentially. By $n = 12$, nearly all digits of precision are lost.

The Hilbert matrix is the canonical example of ill-conditioning:

$$H_{ij} = \frac{1}{i + j - 1} \quad (3.4)$$

For $n = 10$, $\kappa(H) \approx 10^{13}$, meaning the solution retains at most 3 significant figures in double precision.

Chapter 4

Fourier Analysis and Spectral Methods

The Fourier transform decomposes a signal into its frequency components. In computational physics, the **Discrete Fourier Transform** (DFT) and its fast implementation (FFT) are indispensable for signal processing, spectral analysis, and solving differential equations.

4.1 The Discrete Fourier Transform

Given N samples $\{f_k\}_{k=0}^{N-1}$ of a function, the DFT produces N frequency coefficients:

$$\tilde{f}_n = \sum_{k=0}^{N-1} f_k e^{-2\pi i k n / N}, \quad n = 0, 1, \dots, N-1 \quad (4.1)$$

The inverse transform recovers the original samples:

$$f_k = \frac{1}{N} \sum_{n=0}^{N-1} \tilde{f}_n e^{2\pi i k n / N} \quad (4.2)$$

Direct computation of the DFT requires $O(N^2)$ operations. The **Fast Fourier Transform** (FFT), discovered by Cooley and Tukey in 1965, reduces this to $O(N \log N)$ by exploiting the recursive structure of the transform.

```
1 def dft(f):
2     """Compute the DFT of array f (slow, for pedagogy)."""
3     N = len(f)
4     n = np.arange(N)
5     k = n.reshape((N, 1))
6     W = np.exp(-2j * np.pi * k * n / N)
7     return W @ f
8
9 # In practice, use NumPy's optimized FFT:
10 # F = np.fft.fft(signal)
```

4.2 Frequency Resolution and Aliasing

For N samples taken at intervals Δt , the frequency resolution is $\Delta f = 1/(N\Delta t)$ and the maximum resolvable frequency (Nyquist frequency) is:

$$f_{\text{Nyquist}} = \frac{1}{2\Delta t} \quad (4.3)$$

Frequencies above f_{Nyquist} are **aliased**—they appear as spurious lower frequencies. The Nyquist-Shannon sampling theorem states that a signal must be sampled at least twice per period of its highest frequency component to be faithfully represented.

4.3 Power Spectrum

The **power spectrum** $P(f) = |\tilde{f}(f)|^2$ reveals the energy content at each frequency. Peaks in the power spectrum identify dominant periodic components in the signal.

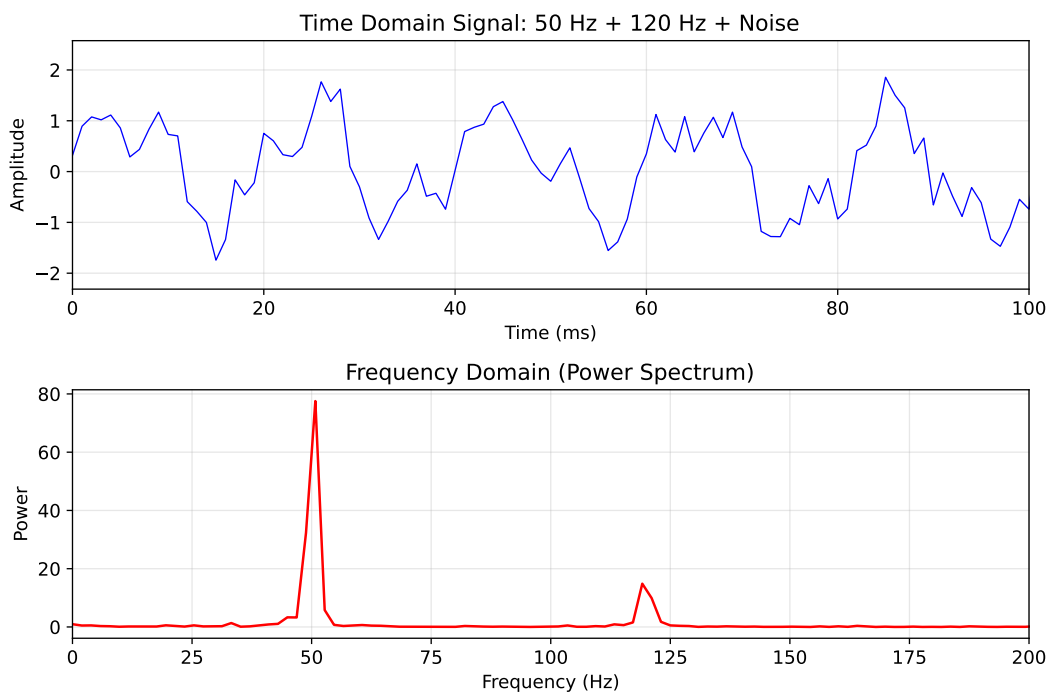


Figure 4.1: Top: a time-domain signal composed of 50 Hz and 120 Hz sinusoids plus Gaussian noise. Bottom: the power spectrum clearly resolves both frequency components above the noise floor.

4.4 Fourier Series and Gibbs Phenomenon

A periodic function $f(t)$ with period T can be expanded as:

$$f(t) = \frac{a_0}{2} + \sum_{n=1}^{\infty} \left[a_n \cos\left(\frac{2\pi nt}{T}\right) + b_n \sin\left(\frac{2\pi nt}{T}\right) \right] \quad (4.4)$$

For a square wave of unit amplitude and period 1:

$$f(t) = \frac{4}{\pi} \sum_{n=1,3,5,\dots}^{\infty} \frac{1}{n} \sin(2\pi nt) \quad (4.5)$$

Truncating this series at $n_{\max}$ terms produces oscillations near discontinuities—the **Gibbs phenomenon**. The overshoot converges to approximately 9% of the jump, regardless of how many terms are included.

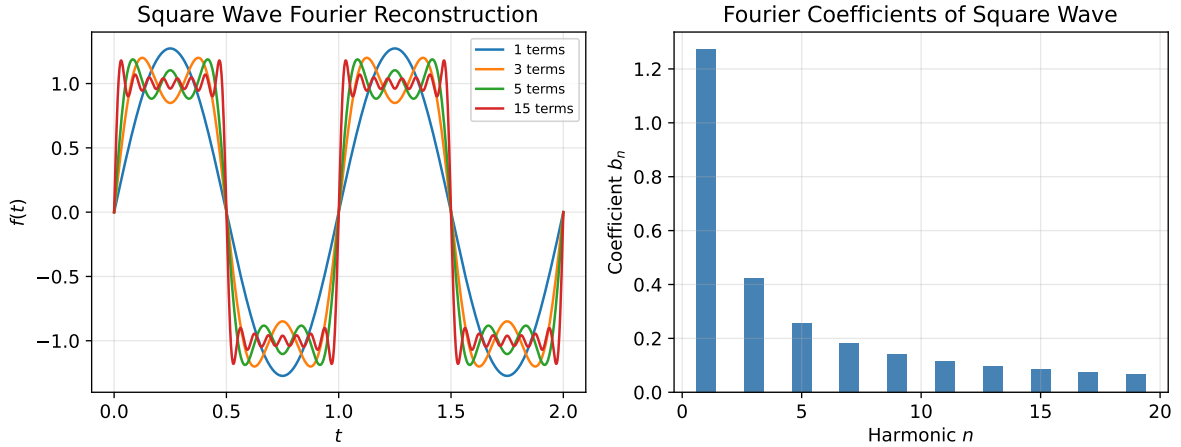


Figure 4.2: Left: Fourier reconstruction of a square wave with 1, 3, 5, and 15 harmonics. The Gibbs overshoot near discontinuities persists even with many terms. Right: Fourier coefficients $b_n = 4/(n\pi)$ decay as $1/n$, characteristic of a function with jump discontinuities.

Chapter 5

Ordinary Differential Equations

Ordinary differential equations (ODEs) govern the time evolution of dynamical systems: planetary orbits, oscillating circuits, population dynamics, and quantum states. We consider initial value problems (IVPs) of the form:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0 \quad (5.1)$$

5.1 Euler's Method

The simplest integrator advances the solution by one step using the derivative at the current point:

$$y_{n+1} = y_n + h f(t_n, y_n) \quad (5.2)$$

Euler's method is **first-order**: $|E| = O(h)$. While conceptually simple, it is rarely used in practice due to its low accuracy and poor stability properties.

5.2 The Runge-Kutta Family

Higher-order methods evaluate f at multiple points within each step to achieve better accuracy. The most widely used is the **fourth-order Runge-Kutta** (RK4) method:

$$k_1 = h f(t_n, y_n) \quad (5.3)$$

$$k_2 = h f(t_n + h/2, y_n + k_1/2) \quad (5.4)$$

$$k_3 = h f(t_n + h/2, y_n + k_2/2) \quad (5.5)$$

$$k_4 = h f(t_n + h, y_n + k_3) \quad (5.6)$$

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (5.7)$$

RK4 is **fourth-order**: $|E| = O(h^4)$, achieving four extra orders of accuracy at the cost of four function evaluations per step (versus one for Euler).

```

1 def rk4_step(f, t, y, h):
2     """Single step of the 4th-order Runge-Kutta method."""
3     k1 = h * f(t, y)
4     k2 = h * f(t + h/2, y + k1/2)
5     k3 = h * f(t + h/2, y + k2/2)
6     k4 = h * f(t + h, y + k3)
7     return y + (k1 + 2*k2 + 2*k3 + k4) / 6

```

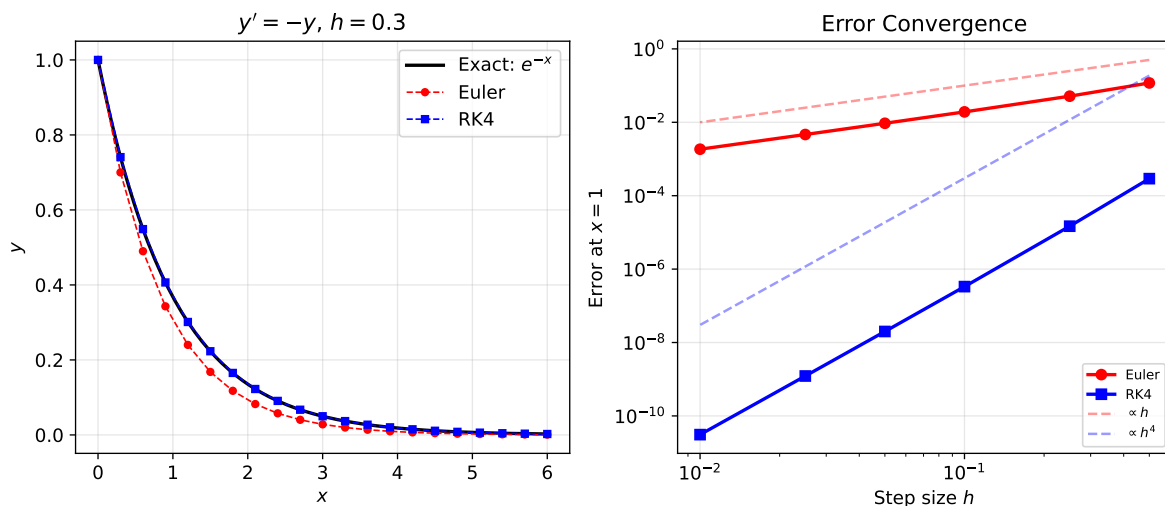


Figure 5.1: Left: Euler and RK4 solutions of $y' = -y$ with $h = 0.3$. Euler drifts noticeably from the exact solution, while RK4 remains indistinguishable from it. Right: error convergence confirms first-order (Euler) and fourth-order (RK4) rates.

5.3 Systems of ODEs

Higher-order ODEs are converted to first-order systems. For example, Newton's second law $m\ddot{x} = F(x, \dot{x}, t)$ becomes:

$$\frac{d}{dt} \begin{pmatrix} x \\ v \end{pmatrix} = \begin{pmatrix} v \\ F(x, v, t)/m \end{pmatrix} \quad (5.8)$$

The same RK4 algorithm applies, with y replaced by a vector.

```

1 def rk4_system(f, t, y, h):
2     """RK4 for a system: y is a numpy array, f returns an array."""
3     k1 = h * f(t, y)
4     k2 = h * f(t + h/2, y + k1/2)
5     k3 = h * f(t + h/2, y + k2/2)
6     k4 = h * f(t + h, y + k3)
7     return y + (k1 + 2*k2 + 2*k3 + k4) / 6

```

5.4 The Damped Driven Oscillator

A canonical system in physics is the damped driven harmonic oscillator:

$$\ddot{x} + \gamma\dot{x} + \omega_0^2 x = F_0 \cos(\omega_d t) \quad (5.9)$$

This system exhibits transient and steady-state behavior, resonance phenomena, and rich phase-space structure.

At long times, the solution approaches a steady state at the driving frequency:

$$x(t) \rightarrow A \cos(\omega_d t - \phi) \quad (5.10)$$

where the amplitude A and phase lag ϕ depend on the driving frequency relative to resonance:

$$A = \frac{F_0}{\sqrt{(\omega_0^2 - \omega_d^2)^2 + \gamma^2 \omega_d^2}}, \quad \tan \phi = \frac{\gamma \omega_d}{\omega_0^2 - \omega_d^2} \quad (5.11)$$

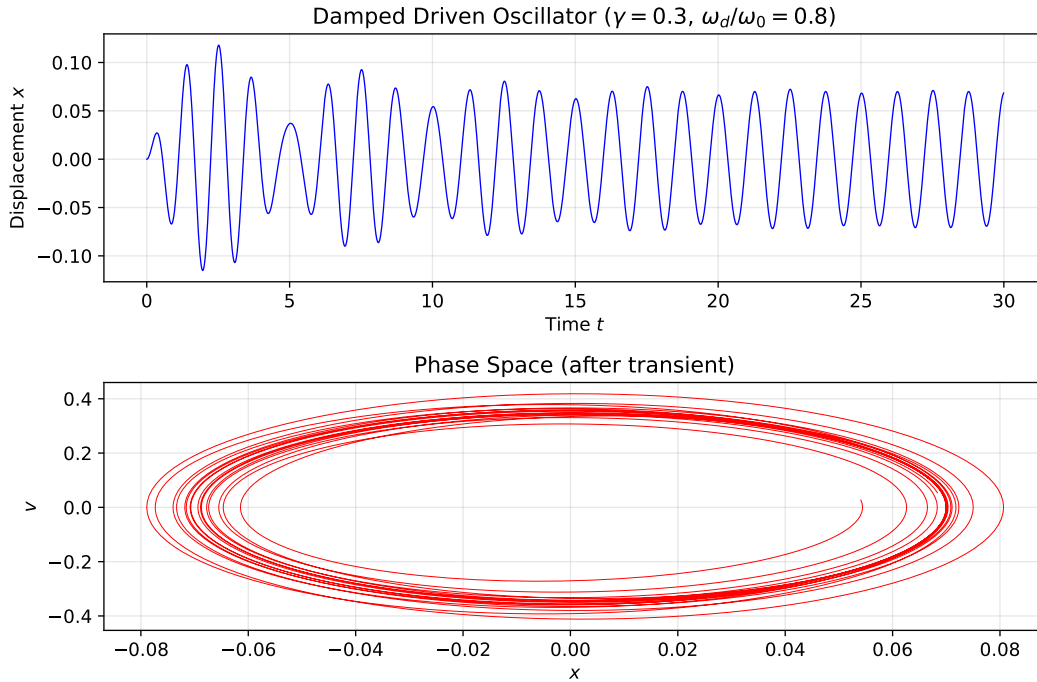


Figure 5.2: Top: displacement versus time for a damped driven oscillator, showing the transient decay into steady-state oscillation. Bottom: phase-space trajectory (x vs v) after the transient has died away, revealing the elliptical limit cycle.

Chapter 6

Orbital Mechanics

The gravitational two-body problem provides an ideal testbed for ODE solvers. The equations are well understood analytically, enabling rigorous validation of numerical methods.

6.1 Newtonian Gravity

A body of mass m orbiting a central mass M obeys:

$$\ddot{\mathbf{r}} = -\frac{GM}{r^3}\mathbf{r} \tag{6.1}$$

In Cartesian coordinates:

$$\ddot{x} = -\frac{GM x}{(x^2 + y^2)^{3/2}} \tag{6.2}$$

$$\ddot{y} = -\frac{GM y}{(x^2 + y^2)^{3/2}} \tag{6.3}$$

The solution is a conic section (ellipse, parabola, or hyperbola) described by:

$$r(\theta) = \frac{p}{1 + e \cos \theta} \tag{6.4}$$

where $p = a(1 - e^2)$ is the semi-latus rectum, a is the semi-major axis, and e is the eccentricity.

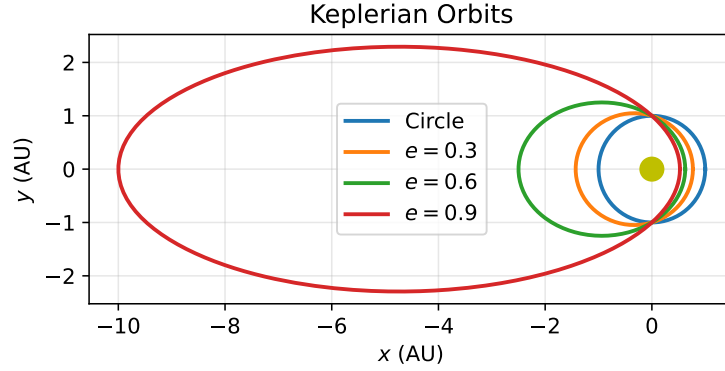


Figure 6.1: Keplerian orbits for various eccentricities. The yellow dot marks the focus (central body). As $e \rightarrow 1$, the orbit becomes increasingly elongated.

6.2 General Relativistic Correction

Einstein's general relativity modifies the gravitational force with a correction term:

$$\ddot{\mathbf{r}} = -\frac{GM}{r^3}\mathbf{r}\left(1 + \frac{\alpha}{r^2}\right) \quad (6.5)$$

where $\alpha = 3GML^2/(m^2c^2)$ depends on the angular momentum L . This correction causes the perihelion of the orbit to precess—a famously confirmed prediction of general relativity.

For Mercury, the predicted precession is 43 arcseconds per century, in precise agreement with observation.

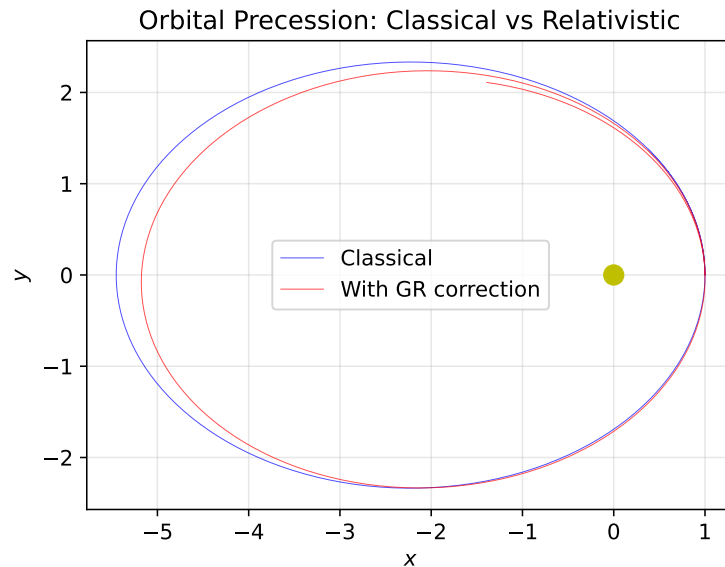


Figure 6.2: Comparison of classical (closed ellipse) and relativistically corrected orbits. The GR correction causes the ellipse to precess, so the orbit does not close. The precession is exaggerated here for visibility.

6.3 Symplectic Integrators

Standard Runge-Kutta methods do not preserve the symplectic structure of Hamiltonian systems, causing energy to drift over long integrations. The **leapfrog** (Störmer-Verlet) method is symplectic and second-order:

$$v_{n+1/2} = v_n + \frac{h}{2}a(x_n) \tag{6.6}$$

$$x_{n+1} = x_n + h v_{n+1/2} \tag{6.7}$$

$$v_{n+1} = v_{n+1/2} + \frac{h}{2}a(x_{n+1}) \tag{6.8}$$

This method conserves a modified energy exactly, making it ideal for long-time orbital integrations.

Chapter 7

Adaptive Methods and Advanced ODE Solvers

Fixed step-size methods waste effort in regions where the solution varies slowly and may miss rapid changes elsewhere. Adaptive methods adjust the step size automatically to maintain a specified error tolerance.

7.1 Step Doubling

The idea is simple: compute the solution from t_n to $t_n + h$ in two ways:

1. One big step of size h , giving y_1
2. Two half-steps of size $h/2$, giving y_2

For a method of order p , Richardson extrapolation gives the error estimate:

$$\epsilon \approx \frac{|y_2 - y_1|}{2^p - 1} \quad (7.1)$$

and a more accurate solution:

$$y_{\text{improved}} = y_2 + \frac{y_2 - y_1}{2^p - 1} \quad (7.2)$$

The step size is adjusted by a factor:

$$\rho = \left(\frac{\delta}{\epsilon} \right)^{1/(p+1)} \quad (7.3)$$

where δ is the target accuracy. If $\rho \geq 1$, the step is accepted and h is increased; if $\rho < 1$, the step is rejected and h is decreased.

```
1 def adaptive_rk4(f, t0, y0, t_end, h0, tol):
2     """RK4 with adaptive step-size control via step doubling."""
3     t, y, h = t0, y0.copy(), h0
```

```
4     ts, ys = [t], [y.copy()]
5
6     while t < t_end:
7         if t + h > t_end:
8             h = t_end - t
9
10        # One full step
11        y1 = rk4_step(f, t, y, h)
12
13        # Two half steps
14        y_mid = rk4_step(f, t, y, h/2)
15        y2 = rk4_step(f, t + h/2, y_mid, h/2)
16
17        # Error estimate (RK4 is order 4)
18        err = np.max(np.abs(y2 - y1)) / 15
19
20        if err < 1e-30:
21            err = 1e-30
22        rho = (tol / err) ** 0.2 # 1/(p+1) = 1/5
23
24        if rho >= 1: # Accept step
25            t += h
26            y = y2 + (y2 - y1) / 15 # Richardson improvement
27            ts.append(t)
28            ys.append(y.copy())
29            h *= min(rho, 2.0) # Don't grow too fast
30        else:
31            h *= max(rho, 0.5) # Don't shrink too fast
32
33    return np.array(ts), np.array(ys)
```

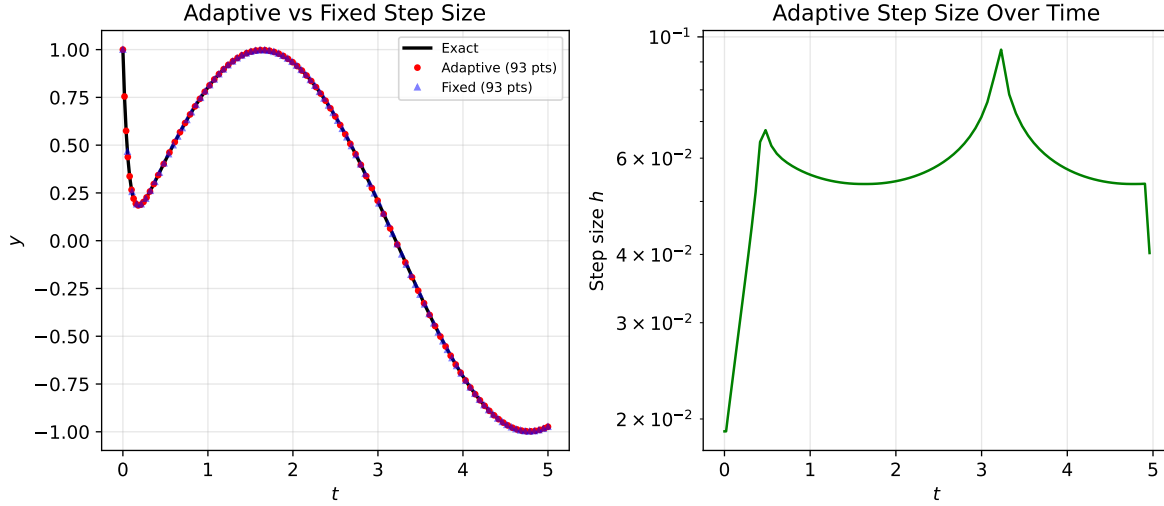


Figure 7.1: Left: adaptive stepping concentrates points where the solution varies rapidly (near $t = 0$ for this stiff-ish problem), while using the same total number of points as the fixed method. Right: the step size $h(t)$ chosen by the adaptive algorithm, showing small steps during the initial transient and large steps during the smooth steady state.

7.2 The Bulirsch-Stoer Method

For problems requiring very high accuracy, the **Bulirsch-Stoer** method combines the modified midpoint method with rational function extrapolation. The idea is:

1. Divide the interval into n sub-steps and integrate with the modified midpoint method
2. Repeat for a sequence of increasing n values: $n = 2, 4, 6, 8, \dots$
3. Extrapolate the results to the $n \rightarrow \infty$ limit using rational function (or polynomial) extrapolation

The **modified midpoint method** for step size H divided into n sub-intervals ($h = H/n$):

$$z_0 = y_0 \tag{7.4}$$

$$z_1 = z_0 + h f(t_0, z_0) \tag{7.5}$$

$$z_{m+1} = z_{m-1} + 2h f(t_0 + mh, z_m), \quad m = 1, \dots, n-1 \tag{7.6}$$

$$y(t_0 + H) \approx \frac{1}{2} [z_n + z_{n-1} + h f(t_0 + H, z_n)] \tag{7.7}$$

The Bulirsch-Stoer method can achieve very high accuracy (12–14 digits) with fewer function evaluations than RK4 for smooth problems.

Chapter 8

Elliptic Partial Differential Equations

Elliptic PDEs describe equilibrium (time-independent) phenomena: electrostatic potentials, steady-state heat conduction, and gravitational fields. The prototype is Laplace's equation.

8.1 Laplace's Equation

Laplace's equation in two dimensions is:

$$\nabla^2 \phi = \frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} = 0 \quad (8.1)$$

Solutions (harmonic functions) satisfy the **mean value property**: the value at any interior point equals the average of values on any surrounding circle.

8.2 Finite Difference Discretization

On a grid with spacing h , the Laplacian is approximated by the five-point stencil:

$$\nabla^2 \phi \approx \frac{\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1} - 4\phi_{i,j}}{h^2} \quad (8.2)$$

Setting this to zero gives the discrete Laplace equation:

$$\phi_{i,j} = \frac{1}{4} (\phi_{i+1,j} + \phi_{i-1,j} + \phi_{i,j+1} + \phi_{i,j-1}) \quad (8.3)$$

This is exactly the mean value property on the grid: each interior value is the average of its four neighbors.

8.3 Iterative Solvers

8.3.1 Jacobi Method

The simplest iterative approach updates all grid points simultaneously using values from the previous iteration:

$$\phi_{i,j}^{(k+1)} = \frac{1}{4} \left(\phi_{i+1,j}^{(k)} + \phi_{i-1,j}^{(k)} + \phi_{i,j+1}^{(k)} + \phi_{i,j-1}^{(k)} \right) \quad (8.4)$$

8.3.2 Gauss-Seidel Method

The Gauss-Seidel method improves convergence by using updated values as soon as they are available:

$$\phi_{i,j}^{(k+1)} = \frac{1}{4} \left(\phi_{i+1,j}^{(k)} + \phi_{i-1,j}^{(k+1)} + \phi_{i,j+1}^{(k)} + \phi_{i,j-1}^{(k+1)} \right) \quad (8.5)$$

This converges roughly twice as fast as Jacobi.

8.3.3 Successive Over-Relaxation (SOR)

SOR accelerates convergence by “over-correcting” each update:

$$\phi_{i,j}^{(k+1)} = (1 - \omega) \phi_{i,j}^{(k)} + \omega \cdot \frac{1}{4} \left(\phi_{i+1,j}^{(k)} + \phi_{i-1,j}^{(k+1)} + \phi_{i,j+1}^{(k)} + \phi_{i,j-1}^{(k+1)} \right) \quad (8.6)$$

The relaxation parameter $\omega \in (1, 2)$ controls the extrapolation. The optimal value for an $N \times N$ grid is approximately:

$$\omega_{\text{opt}} = \frac{2}{1 + \sin(\pi/N)} \approx 2 - \frac{2\pi}{N} \quad (8.7)$$

With optimal ω , SOR converges in $O(N)$ iterations instead of $O(N^2)$ for Gauss-Seidel.

```

1 def laplace_sor(phi, omega, tol=1e-6, max_iter=10000):
2     """Solve Laplace's equation using SOR."""
3     N = phi.shape[0]
4     for iteration in range(max_iter):
5         max_change = 0.0
6         for i in range(1, N-1):
7             for j in range(1, N-1):
8                 old = phi[i, j]
9                 gauss_seidel = 0.25 * (phi[i+1,j] + phi[i-1,j]
10                                     + phi[i,j+1] + phi[i,j-1])
11                 phi[i, j] = (1-omega)*old + omega*gauss_seidel
12                 max_change = max(max_change, abs(phi[i,j] - old))
13     if max_change < tol:
14         print(f"Converged in {iteration+1} iterations")
15         return phi
16     print("Warning: did not converge")
17     return phi

```

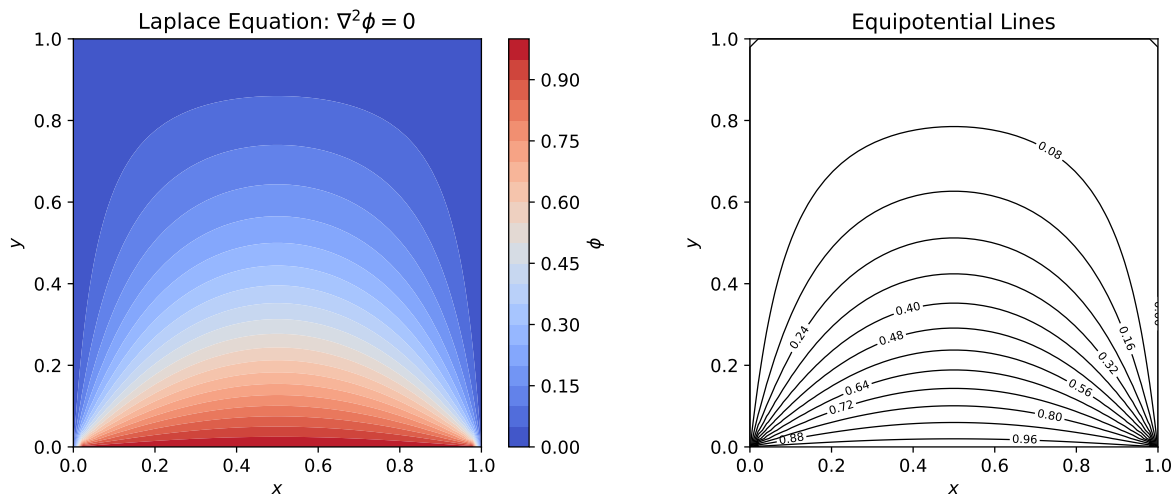


Figure 8.1: Left: filled contour plot of the solution to Laplace's equation with $\phi = 1$ on the top boundary and $\phi = 0$ on the other three sides. Right: equipotential lines showing the smooth interpolation from the heated boundary.

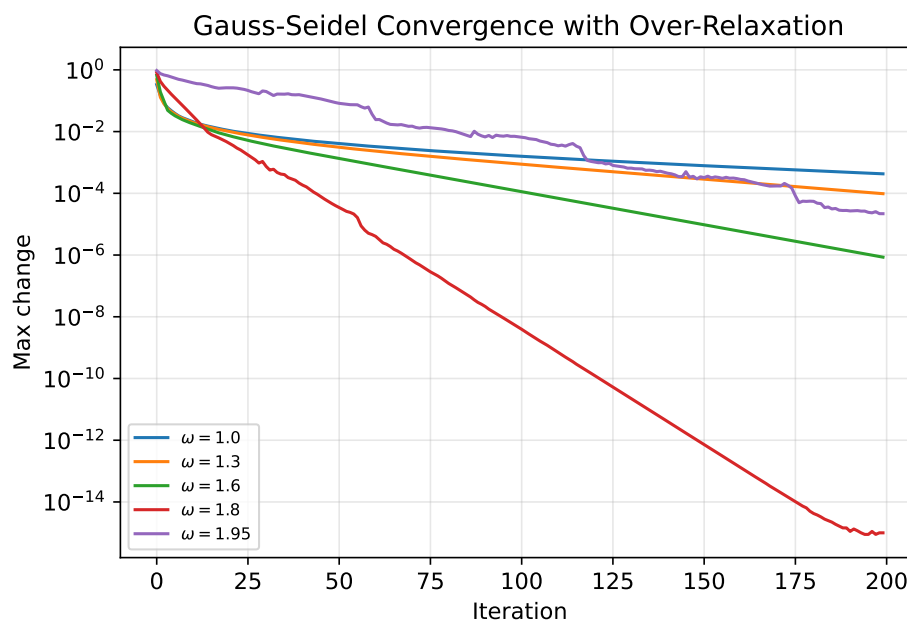


Figure 8.2: Convergence of the Gauss-Seidel iteration for different values of the relaxation parameter ω . Under-relaxation ($\omega < 1$) converges slowly. Over-relaxation near the optimal value ($\omega \approx 1.8$ for this grid) converges much faster, but ω too close to 2 causes oscillation.

Chapter 9

Hyperbolic Partial Differential Equations

Hyperbolic PDEs describe wave propagation and transport phenomena. Unlike elliptic PDEs, they involve time evolution and can develop discontinuities (shocks) even from smooth initial data.

9.1 The Advection Equation

The simplest hyperbolic PDE is the linear advection equation:

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0 \quad (9.1)$$

The exact solution is a rightward-traveling wave: $u(x, t) = u_0(x - ct)$. This equation serves as a testbed for numerical schemes.

9.2 Finite Difference Schemes

9.2.1 FTCS Scheme (Unstable!)

Forward-Time, Central-Space:

$$u_j^{n+1} = u_j^n - \frac{c\Delta t}{2\Delta x}(u_{j+1}^n - u_{j-1}^n) \quad (9.2)$$

This scheme is **unconditionally unstable** for the advection equation—a von Neumann stability analysis shows that all modes grow. It should never be used for hyperbolic problems.

9.2.2 Lax-Friedrichs Scheme

Replace u_j^n with the average of its neighbors:

$$u_j^{n+1} = \frac{1}{2}(u_{j+1}^n + u_{j-1}^n) - \frac{c\Delta t}{2\Delta x}(u_{j+1}^n - u_{j-1}^n) \quad (9.3)$$

This is stable under the CFL condition $|c|\Delta t/\Delta x \leq 1$, but introduces numerical diffusion.

9.2.3 Leapfrog Scheme

$$u_j^{n+1} = u_j^{n-1} - \frac{c\Delta t}{\Delta x}(u_{j+1}^n - u_{j-1}^n) \quad (9.4)$$

Second-order in both space and time, non-dissipative, but supports a spurious “computational mode” that must be controlled.

9.2.4 Lax-Wendroff Scheme

$$u_j^{n+1} = u_j^n - \frac{c\Delta t}{2\Delta x}(u_{j+1}^n - u_{j-1}^n) + \frac{c^2\Delta t^2}{2\Delta x^2}(u_{j+1}^n - 2u_j^n + u_{j-1}^n) \quad (9.5)$$

Second-order accurate and stable under CFL, with less dissipation than Lax-Friedrichs but some dispersion. It is derived by Taylor-expanding $u(x, t + \Delta t)$ to second order and replacing time derivatives with spatial derivatives using the PDE.

9.3 The CFL Condition

The **Courant-Friedrichs-Lewy** condition states that the numerical domain of dependence must contain the physical domain of dependence:

$$\text{CFL number} = \frac{|c|\Delta t}{\Delta x} \leq 1 \quad (9.6)$$

Violating this condition causes catastrophic instability in explicit schemes.

9.4 Burgers’ Equation

The inviscid Burgers’ equation is the simplest nonlinear hyperbolic PDE:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = 0 \quad (9.7)$$

This is a conservation law $u_t + f(u)_x = 0$ with flux $f(u) = u^2/2$. Characteristics carry constant values of u :

$$\frac{dx}{dt} = u \quad (9.8)$$

When characteristics cross, a **shock** forms—a discontinuity in the solution that develops from smooth initial data. The shock speed is given by the Rankine-Hugoniot condition:

$$s = \frac{f(u_L) - f(u_R)}{u_L - u_R} = \frac{u_L + u_R}{2} \quad (9.9)$$

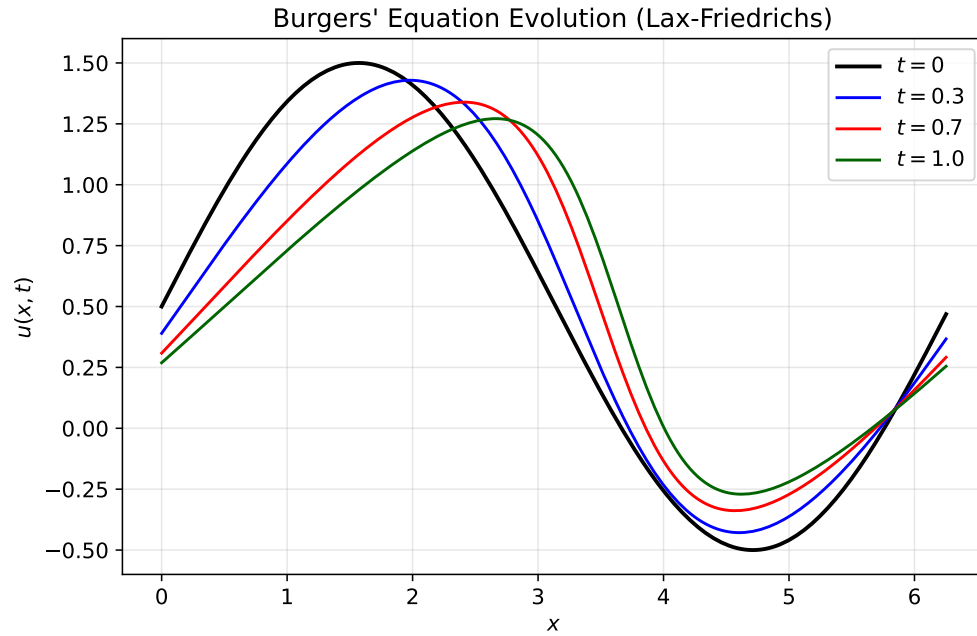


Figure 9.1: Evolution of Burgers' equation with initial condition $u(x, 0) = \sin(x) + 0.5$ solved using the Lax-Friedrichs scheme. The wave steepens as faster portions overtake slower ones, eventually forming a shock.

Chapter 10

Monte Carlo Methods

Monte Carlo methods use random sampling to solve problems that may be deterministic in principle but intractable by standard numerical methods. They are essential in statistical physics, radiative transfer, and high-dimensional integration.

10.1 Monte Carlo Integration

To evaluate $I = \int_a^b f(x) dx$, draw N random points $x_1, \dots, x_N$ uniformly from $[a, b]$ and estimate:

$$I \approx \frac{b-a}{N} \sum_{i=1}^N f(x_i) \quad (10.1)$$

The error decreases as $O(1/\sqrt{N})$, regardless of dimensionality. This makes Monte Carlo the method of choice for high-dimensional integrals, where deterministic quadrature suffers from the curse of dimensionality.

For a d -dimensional integral over the unit hypercube:

$$\int_{[0,1]^d} f(\mathbf{x}) d\mathbf{x} \approx \frac{1}{N} \sum_{i=1}^N f(\mathbf{x}_i) \quad (10.2)$$

The variance of the estimator is:

$$\text{Var}(\hat{I}) = \frac{\text{Var}(f)}{N} = \frac{1}{N} [\langle f^2 \rangle - \langle f \rangle^2] \quad (10.3)$$

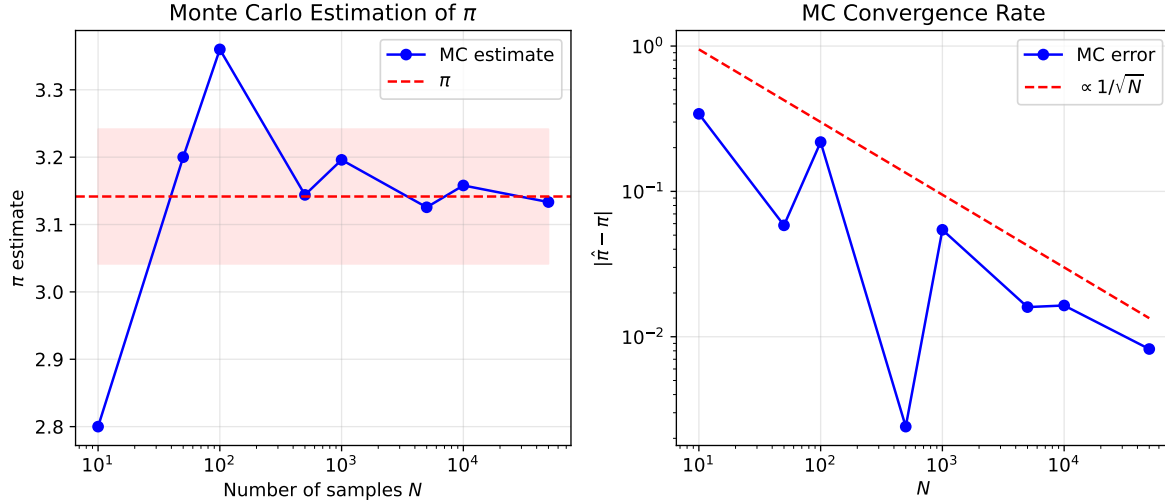


Figure 10.1: Left: Monte Carlo estimation of π by sampling random points in the unit square and counting those inside the quarter-circle. The estimate converges to π as N increases. Right: the error scales as $1/\sqrt{N}$, characteristic of Monte Carlo.

10.2 Importance Sampling

The variance can be reduced dramatically by sampling from a distribution $p(x)$ that concentrates points where $|f(x)|$ is large:

$$I = \int_a^b f(x) dx = \int_a^b \frac{f(x)}{p(x)} p(x) dx \approx \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)} \quad (10.4)$$

where $x_i \sim p(x)$. The optimal choice $p^*(x) \propto |f(x)|$ gives zero variance, but is impractical since it requires knowing the integral. In practice, $p(x)$ is chosen to roughly match the shape of $|f(x)|$.

10.3 Random Walks and Transport

Monte Carlo methods naturally simulate stochastic processes. A **random walk** on a lattice or in continuous space models diffusion, polymer conformations, and particle transport.

In radiative transfer, photon transport through a scattering medium is modeled by:

1. Draw a random optical depth $\tau = -\ln(\xi)$ to the next interaction (ξ uniform on $[0, 1]$)
2. Move the photon a physical distance $\ell = \tau/\kappa$ where κ is the opacity
3. At the interaction point, draw a new direction from the scattering phase function
4. Repeat until the photon escapes or is absorbed

For isotropic scattering, the new direction cosine is drawn uniformly from $[-1, 1]$. The mean number of scatterings before escape from a slab of optical depth $\tau_{\max}$ scales as $\tau_{\max}^2$ for a random walk.

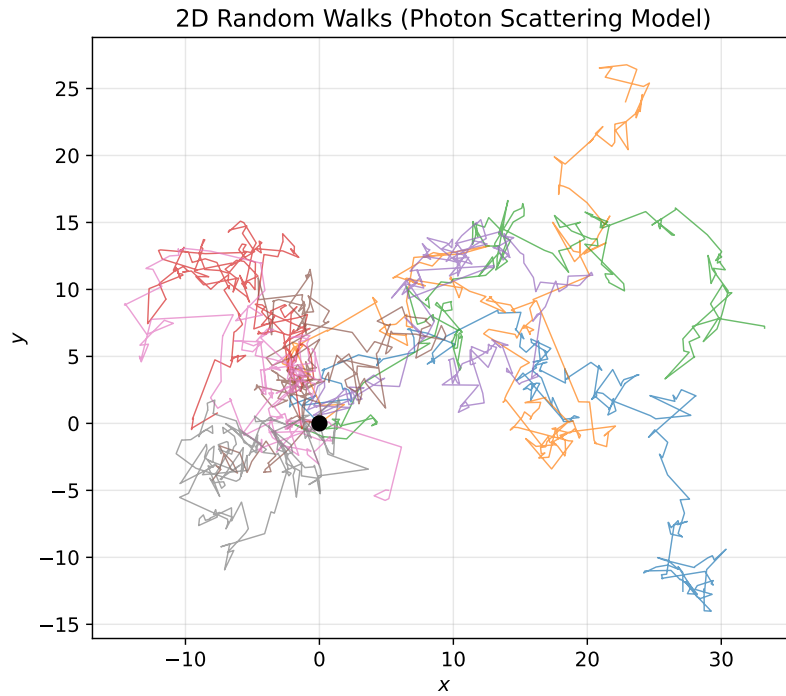


Figure 10.2: Eight independent 2D random walks with exponentially distributed step lengths, modeling photon scattering. Despite starting from the same point, the walkers diffuse to very different final positions, illustrating the stochastic nature of the process.

Chapter 11

Markov Chain Monte Carlo and Statistical Mechanics

When we need to sample from a complex probability distribution $P(\mathbf{x})$ that cannot be directly sampled, **Markov Chain Monte Carlo** (MCMC) constructs a random walk through state space whose stationary distribution is P .

11.1 The Boltzmann Distribution

In statistical mechanics, the probability of a microstate with energy E at temperature T is:

$$P(E) = \frac{1}{Z} e^{-E/k_B T} \quad (11.1)$$

where $Z = \sum_{\text{states}} e^{-E/k_B T}$ is the partition function. Computing Z directly requires summing over all states—an exponentially large number for most systems.

MCMC sidesteps the partition function entirely: we only need *ratios* of probabilities, which cancel Z .

11.2 The Metropolis Algorithm

The Metropolis algorithm (1953) generates a Markov chain $\{x_0, x_1, x_2, \dots\}$ whose distribution converges to the target $P(x)$:

1. Start in state x_n
2. Propose a new state x' from a symmetric proposal distribution $q(x'|x_n)$
3. Compute the acceptance ratio:

$$\alpha = \frac{P(x')}{P(x_n)} = e^{-(E(x') - E(x_n))/k_B T} \quad (11.2)$$

4. Accept $x' \rightarrow x_{n+1}$ with probability $\min(1, \alpha)$:

- If $\alpha \geq 1$ (energy decreased): always accept
 - If $\alpha < 1$ (energy increased): accept with probability α
5. If rejected, set $x_{n+1} = x_n$

Detailed balance guarantees convergence to P :

$$P(x) T(x \rightarrow x') = P(x') T(x' \rightarrow x) \quad (11.3)$$

```

1 def metropolis(energy_func, x0, T, n_steps, proposal):
2     """Metropolis algorithm for sampling Boltzmann distribution."""
3     x = x0
4     samples = [x]
5     E = energy_func(x)
6
7     for _ in range(n_steps):
8         x_new = proposal(x)
9         E_new = energy_func(x_new)
10        dE = E_new - E
11
12        if dE <= 0 or np.random.random() < np.exp(-dE / T):
13            x = x_new
14            E = E_new
15            samples.append(x)
16
17    return np.array(samples)
    
```

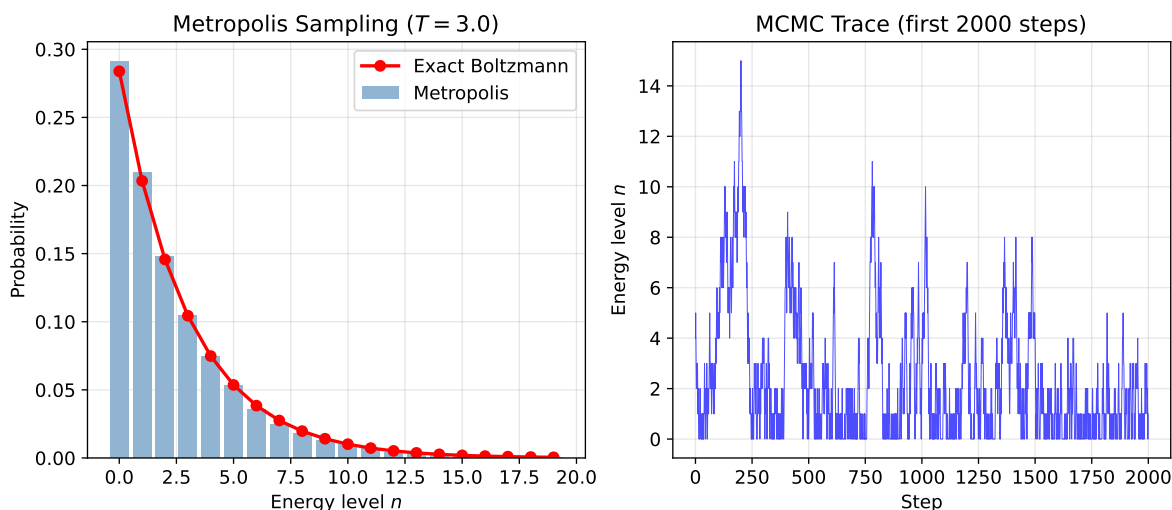


Figure 11.1: Left: histogram of energy levels sampled by the Metropolis algorithm (blue bars) compared to the exact Boltzmann distribution (red circles) at $T = 3$. The agreement validates the algorithm. Right: the MCMC trace shows the random walk through energy space, with occasional excursions to higher levels.

11.3 Quantum Systems at Finite Temperature

Consider a quantum particle in a 3D infinite square well with energy:

$$E_{n_x, n_y, n_z} = \frac{\pi^2 \hbar^2}{2mL^2} (n_x^2 + n_y^2 + n_z^2), \quad n_x, n_y, n_z = 1, 2, 3, \dots \quad (11.4)$$

At finite temperature T , the occupation probability of state (n_x, n_y, n_z) is given by the Boltzmann factor. The Metropolis algorithm samples these states by proposing changes $n_i \rightarrow n_i \pm 1$ and accepting according to $e^{-\Delta E/k_B T}$.

At low temperature, only the ground state and first few excited states are populated. As T increases, higher states become accessible, and the energy distribution broadens.

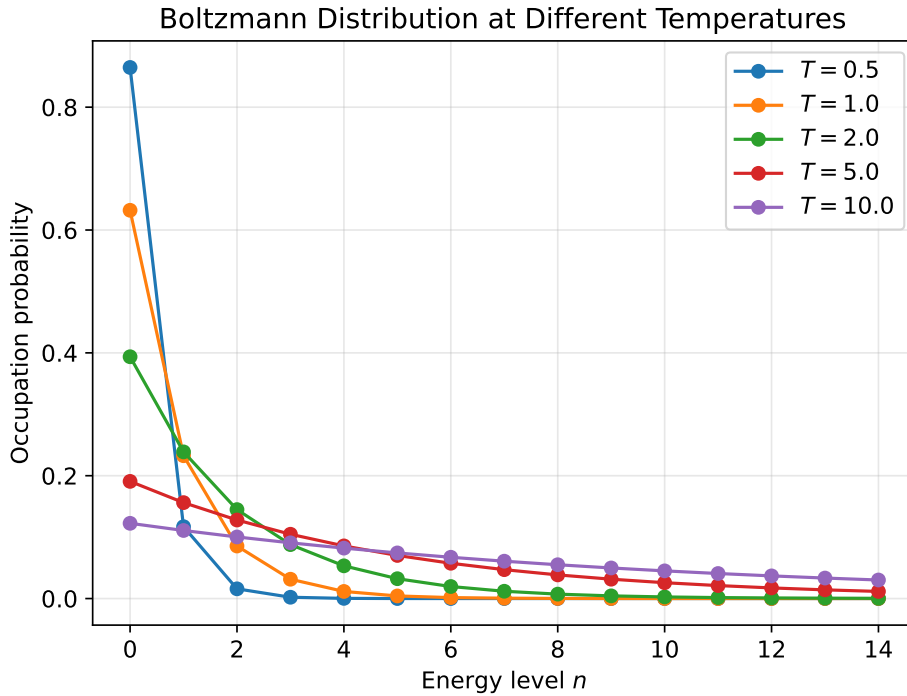


Figure 11.2: Boltzmann occupation probabilities at different temperatures. At low T , the ground state dominates. At high T , many states are populated with nearly equal probability, approaching the classical equipartition limit.

11.4 Practical Considerations

Several factors affect MCMC performance:

Burn-in: The initial samples are biased by the starting state and should be discarded. Typically, the first 10–20% of samples are removed.

Autocorrelation: Successive samples are correlated. The *effective sample size* is $N_{\text{eff}} = N / (1 + 2 \sum_k \rho_k)$ where ρ_k is the autocorrelation at lag k . Only every τ_{corr} -th sample is effectively independent.

Proposal scale: If the proposal steps are too small, the chain explores slowly (high acceptance, high correlation). If too large, most proposals are rejected. The optimal acceptance rate for Gaussian proposals in high dimensions is approximately 23%.

Convergence diagnostics: Run multiple chains from different starting points and compare their distributions. The Gelman-Rubin statistic $\hat{R}$ compares within-chain and between-chain variance; values near 1 indicate convergence.

Appendix A

Mathematical Reference

A.1 Taylor Series

The Taylor expansion of $f(x)$ about $x = a$:

$$f(x) = f(a) + f'(a)(x - a) + \frac{f''(a)}{2!}(x - a)^2 + \frac{f'''(a)}{3!}(x - a)^3 + \dots \quad (\text{A.1})$$

Key applications in numerical methods:

$$f(x + h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + O(h^3) \quad (\text{A.2})$$

$$f(x - h) = f(x) - hf'(x) + \frac{h^2}{2}f''(x) + O(h^3) \quad (\text{A.3})$$

Useful finite difference formulas:

$$f'(x) = \frac{f(x + h) - f(x - h)}{2h} + O(h^2) \quad (\text{central difference}) \quad (\text{A.4})$$

$$f''(x) = \frac{f(x + h) - 2f(x) + f(x - h))}{h^2} + O(h^2) \quad (\text{second derivative}) \quad (\text{A.5})$$

A.2 Linear Algebra Essentials

Eigenvalue decomposition: $A\mathbf{v} = \lambda\mathbf{v}$

Condition number: $\kappa(A) = \|A\| \cdot \|A^{-1}\| = |\lambda_{\max}|/|\lambda_{\min}|$ (for symmetric A)

Matrix norms: $\|A\|_2 = \sigma_{\max}(A)$ (largest singular value)

A.3 Probability and Statistics

Central Limit Theorem: For N i.i.d. samples with mean μ and variance σ^2 :

$$\bar{X}_N = \frac{1}{N} \sum_{i=1}^N X_i \xrightarrow{d} \mathcal{N}\left(\mu, \frac{\sigma^2}{N}\right) \quad (\text{A.6})$$

This is why Monte Carlo errors scale as $1/\sqrt{N}$.

Law of Large Numbers:

$$\bar{X}_N \xrightarrow{a.s.} \mu \quad \text{as } N \rightarrow \infty \quad (\text{A.7})$$

A.4 Useful Integrals

$$\int_0^\pi \sin(x) dx = 2 \quad (\text{A.8})$$

$$\int_0^\infty e^{-x^2} dx = \frac{\sqrt{\pi}}{2} \quad (\text{A.9})$$

$$\int_0^\infty x^{n-1} e^{-x} dx = \Gamma(n) = (n-1)! \quad \text{for } n \in \mathbb{Z}^+ \quad (\text{A.10})$$

$$\int_{-\infty}^\infty e^{-ax^2+bx} dx = \sqrt{\frac{\pi}{a}} \exp\left(\frac{b^2}{4a}\right) \quad (\text{A.11})$$

A.5 Physical Constants

Constant	Symbol	Value
Speed of light	c	$2.998 \times 10^8 \text{ m/s}$
Gravitational constant	G	$6.674 \times 10^{-11} \text{ m}^3\text{kg}^{-1}\text{s}^{-2}$
Planck's constant	$\hbar$	$1.055 \times 10^{-34} \text{ J}\cdot\text{s}$
Boltzmann's constant	k_B	$1.381 \times 10^{-23} \text{ J/K}$
Electron mass	m_e	$9.109 \times 10^{-31} \text{ kg}$
Solar mass	$M_\odot$	$1.989 \times 10^{30} \text{ kg}$